

DP Computer Science: B 4.1.3 Construct and Apply Linked Lists (Python) (HL Only)

1. Core Concepts & Learning Objectives

Overview

This section focuses on the practical skill of constructing and applying various types of linked lists in Python for Higher Level (HL) students. The lessons cover:

- **Singly Linked Lists (SLL):** Node creation, list construction, appending, traversal, insertion, and deletion
 - **Doubly Linked Lists (DLL):** Node structure with `prev` and `next` pointers, append, and traversal in both directions
 - **Circular Linked Lists (CLL):** Structure, append, traversal, and search operations
-

Key Vocabulary

Term	Definition
Node	A building block of a linked list containing data and pointer(s)
Head	A reference to the first node in the list
Tail	A reference to the last node in the list
None	Python's equivalent of null; indicates the end of a list
Singly Linked List (SLL)	Each node points to the next node only
Doubly Linked List (DLL)	Each node points to both the next and previous nodes
Circular Linked List (CLL)	The last node points back to the head
Traversal	The process of visiting each node in a linked list

Term	Definition
Garbage Collection	Automatic memory management in Python that reclaims unreferenced objects

ATL Elements

Skill	Description
Thinking	Algorithmic thinking, problem-solving, critical thinking, adaptation, comparative thinking
Communication	Articulating logic, explaining reference manipulation
Self-Management	Structuring code logically, managing complexity

2. Lesson 1: Singly Linked Lists — Node and List Creation, Appending, and Traversal

Learning Intentions

By the end of this lesson, students will be able to:

- Define and implement a **Node** class for a singly linked list in Python
- Define and implement a basic **SinglyLinkedList** class structure, including a `head` reference
- Implement an **append** method to add new nodes to the end of a singly linked list
- Implement a **traverse** (or display) method to iterate through and print the elements of a singly linked list

The Node Class

```
python
```

```

class Node:
    """A node in a singly linked list."""
    def __init__(self, data):
        self.data = data          # The data stored in the node
        self.next = None         # Reference to the next node (None by default)

    def __str__(self):
        return str(self.data)

```

The SinglyLinkedList Class

python

```

class SinglyLinkedList:
    """A singly linked list."""
    def __init__(self):
        self.head = None         # Reference to the first node (None if empty)

    def is_empty(self):
        """Check if the list is empty."""
        return self.head is None

```

Appending Nodes

python

```

class SinglyLinkedList:
    # ... (previous code)

    def append(self, data):
        """Add a new node with the given data to the end of the list."""
        new_node = Node(data)

        if self.head is None:
            # If the list is empty, the new node becomes the head
            self.head = new_node
        else:
            # Traverse to the last node
            current = self.head
            while current.next is not None:

```

```
        current = current.next
    # Link the last node to the new node
    current.next = new_node
```

Traversing the List

python

```
class SinglyLinkedList:
    # ... (previous code)

    def display(self):
        """Traverse and print all elements in the list."""
        if self.head is None:
            print("Empty list")
            return

        current = self.head
        elements = []
        while current is not None:
            elements.append(str(current.data))
            current = current.next
        print(" -> ".join(elements))
```

Complete Example

python

```
# Create a new singly linked list
my_list = SinglyLinkedList()

# Append some nodes
my_list.append(10)
my_list.append(20)
my_list.append(30)

# Display the list
my_list.display() # Output: 10 -> 20 -> 30
```

3. Lesson 2: Singly Linked Lists — Insertion and Deletion

Learning Intentions

By the end of this lesson, students will be able to:

- Implement a method to **insert** a new node at the **beginning** of a singly linked list
 - Implement a method to **insert** a new node **after a given existing node**
 - Implement a method to **delete** a node with a specific key (value) from a singly linked list
-

Insertion at the Beginning

python

```
class SinglyLinkedList:
    # ... (previous code)

    def insert_at_beginning(self, data):
        """Insert a new node at the beginning of the list."""
        new_node = Node(data)
        new_node.next = self.head    # New node points to old head
        self.head = new_node        # New node becomes the head
```

Insertion After a Given Node

python

```
class SinglyLinkedList:
    # ... (previous code)

    def insert_after_node(self, target_data, new_data):
        """Insert a new node after the first node containing target_data."""
        current = self.head

        # Find the node with the target data
        while current is not None and current.data != target_data:
            current = current.next
```

```

if current is None:
    print(f"Node with data {target_data} not found.")
    return

# Insert the new node after the found node
new_node = Node(new_data)
new_node.next = current.next
current.next = new_node

```

Deletion of a Node

python

```

class SinglyLinkedList:
    # ... (previous code)

    def delete_node(self, key):
        """Delete the first node containing the given key."""
        # If the list is empty
        if self.head is None:
            print("List is empty.")
            return

        # If the node to delete is the head
        if self.head.data == key:
            self.head = self.head.next
            return

        # Traverse to find the node before the one to delete
        current = self.head
        while current.next is not None and current.next.data != key:
            current = current.next

        # If the key was not found
        if current.next is None:
            print(f"Key {key} not found.")
            return

```

```
# Delete the node by bypassing it
current.next = current.next.next
```

Complete Insertion/Deletion Example

python

```
my_list = SinglyLinkedList()
my_list.append(10)
my_list.append(20)
my_list.append(30)

my_list.insert_at_beginning(5)
my_list.display() # Output: 5 -> 10 -> 20 -> 30

my_list.insert_after_node(20, 25)
my_list.display() # Output: 5 -> 10 -> 20 -> 25 -> 30

my_list.delete_node(20)
my_list.display() # Output: 5 -> 10 -> 25 -> 30
```

4. Lesson 3: Doubly Linked Lists

Learning Intentions

By the end of this lesson, students will be able to:

- Understand DLL node structure (data , next , prev)
 - Implement `DLLNode` and `DoublyLinkedList` classes
 - Implement **append** for a doubly linked list
 - Implement **forward** and **backward** traversal
-

The DLLNode Class

python

```
class DLLNode:
    """A node in a doubly linked list."""
    def __init__(self, data):
        self.data = data
        self.prev = None    # Reference to the previous node
        self.next = None    # Reference to the next node

    def __str__(self):
        return str(self.data)
```

The DoublyLinkedList Class

python

```
class DoublyLinkedList:
    """A doubly linked list."""
    def __init__(self):
        self.head = None    # Reference to the first node
        self.tail = None    # Reference to the last node

    def is_empty(self):
        return self.head is None

    def append(self, data):
        """Add a new node to the end of the doubly linked list."""
        new_node = DLLNode(data)

        if self.head is None:
            # Empty List: new node is both head and tail
            self.head = new_node
            self.tail = new_node
        else:
            # Link the new node to the current tail
            self.tail.next = new_node
            new_node.prev = self.tail
            self.tail = new_node
```

Forward Traversal

python

```
class DoublyLinkedList:
    # ... (previous code)

    def traverse_forward(self):
        """Traverse the list from head to tail."""
        if self.head is None:
            print("Empty list")
            return

        current = self.head
        elements = []
        while current is not None:
            elements.append(str(current.data))
            current = current.next
        print(" -> ".join(elements))
```

Backward Traversal

python

```
class DoublyLinkedList:
    # ... (previous code)

    def traverse_backward(self):
        """Traverse the list from tail to head."""
        if self.tail is None:
            print("Empty list")
            return

        current = self.tail
        elements = []
        while current is not None:
            elements.append(str(current.data))
            current = current.prev
        print(" <- ".join(elements))
```

Complete DLL Example

python

```
dll = DoublyLinkedList()
dll.append(10)
dll.append(20)
dll.append(30)

dll.traverse_forward()    # Output: 10 -> 20 -> 30
dll.traverse_backward()  # Output: 30 <- 20 <- 10
```

5. Lesson 4: Circular Linked Lists and Search

Learning Intentions

By the end of this lesson, students will be able to:

- Understand the concept of a **circular singly linked list (CSLL)**
- Adapt SLL for **append** and **traversal**
- Understand **linear search** on linked lists
- Discuss **applications** of linked lists

Circular Singly Linked List (CSLL)

python

```
class CSLLNode:
    """A node in a circular singly linked list."""
    def __init__(self, data):
        self.data = data
        self.next = None

class CircularSinglyLinkedList:
    """A circular singly linked list."""
    def __init__(self):
        self.head = None

    def append(self, data):
```

```

"""Add a new node to the circular list."""
new_node = CSLLNode(data)

if self.head is None:
    # Empty List: new node points to itself
    self.head = new_node
    new_node.next = new_node
else:
    # Traverse to the Last node (the one pointing to head)
    current = self.head
    while current.next != self.head:
        current = current.next
    # Insert the new node
    current.next = new_node
    new_node.next = self.head

```

Traversing a CSLL

python

```

class CircularSinglyLinkedList:
    # ... (previous code)

    def display(self):
        """Traverse and print all elements in the circular list."""
        if self.head is None:
            print("Empty list")
            return

        elements = []
        current = self.head
        while True:
            elements.append(str(current.data))
            current = current.next
            if current == self.head:
                break
        print(" -> ".join(elements) + " -> (back to head)")

```

Searching in a Linked List

python

```
class SinglyLinkedList:
    # ... (previous code)

    def search(self, key):
        """Search for a node containing the given key."""
        current = self.head
        index = 0
        while current is not None:
            if current.data == key:
                print(f"Found {key} at position {index}")
                return index
            current = current.next
            index += 1
        print(f"{key} not found.")
        return -1
```

CSLL Example

python

```
csll = CircularSinglyLinkedList()
csll.append(10)
csll.append(20)
csll.append(30)

csll.display() # Output: 10 -> 20 -> 30 -> (back to head)
```

6. Applications of Linked Lists

Singly Linked Lists

Application	Description
Simple Lists	Lists where forward traversal is sufficient

Application	Description
Stacks	LIFO data structures (push/pop at head)
Queues	FIFO data structures (enqueue at tail, dequeue at head)

Doubly Linked Lists

Application	Description
Music Playlists	Need to go forward and backward
Browser History	Back and forward navigation
LRU Cache	Need to move nodes to the front or back

Circular Linked Lists

Application	Description
Round-Robin Scheduling	Processes take turns in a cycle
Multiplayer Games	Players take turns in a loop
Buffer Management	Continuous loops for streaming data

7. Worksheets

Worksheet	Topic
Worksheet 1	SLL — Node creation, append, traversal
Worksheet 2	SLL — Insertion and deletion
Worksheet 3	DLL — Structure, append, traversal

8. Summary: Key Takeaways

Structure	Key Feature	Best For
Singly Linked List	One-way traversal	Simple lists, stacks, queues
Doubly Linked List	Two-way traversal	Playlists, browser history
Circular Linked List	Loop (no NULL)	Round-robin, continuous loops

text

KEY REMINDERS

✓

SLL: Node(data, next)

✓

DLL: Node(data, prev, next)

✓

CSLL: Last node points to head

✓

Append requires traversal (except with tail)

✓

Insertion/Deletion require careful pointer updates

✓

Python uses None (not null)

✓

Draw diagrams to understand pointer changes

"Building linked lists from scratch teaches us how data structures work at a fundamental level—understanding pointers, references, and memory management is key to mastering computer science."